

# The Art of Computer Programming 3

## Lecture Notes

|            |                                               |    |
|------------|-----------------------------------------------|----|
| Lecture 1. | Computable and uncomputable functions         | 2  |
| Lecture 2. | Decidable and enumerable languages            | 6  |
| Lecture 3. | Turing machines                               | 12 |
| Lecture 4. | From calculating machines to modern computers | 20 |

# Lecture 1. Computable and uncomputable functions

## 1.1 Alphabets and words

At the low level, information inside computers is represented by 0s and 1s. Every program takes as input a finite sequence of bits and returns another such sequence, and all computation as we know it, in the end, is manipulation of bit-strings. Our formalisation begins here.

**Definition 1** (Alphabets and words). An **alphabet**  $\Sigma$  is a non-empty finite set of symbols. A **word** (or **string**) over  $\Sigma$  is a finite sequence of symbols  $w = a_1 a_2 \dots a_n$ , where each  $a_i \in \Sigma$ . The unique sequence of length zero is the **empty word**, denoted  $\varepsilon$ . The set of all finite words over  $\Sigma$  is denoted  $\Sigma^*$ .

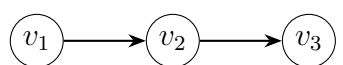
Given two words  $u = a_1 \dots a_n$  and  $v = b_1 \dots b_m$ , their **concatenation** is  $uv = a_1 \dots a_n b_1 \dots b_m$ . Concatenation is associative:  $(uv)w = u(vw)$ , and  $\varepsilon$  is the two-sided identity:  $u\varepsilon = \varepsilon u = u$ .<sup>1</sup>

**Binary encoding.** The ‘canonical’ alphabet for computers is the *binary alphabet*  $\{0, 1\}$ . Words in any other alphabet  $\Sigma$  can be *encoded* by binary words as follows. We fix a length  $k$  such that  $2^k \geq |\Sigma|$ , and assign a distinct  $k$ -bit binary word to each symbol  $a_i$ . For example, if  $\Sigma = \{a, b, c\}$ , we can choose  $k = 2$  and map

$$a \mapsto 00, \quad b \mapsto 01, \quad c \mapsto 10.$$

A word over  $\Sigma$  is then encoded by replacing each symbol with its code: for instance,  $cab$  becomes 100001. Since every symbol uses exactly 2 bits (or, in general,  $k$ ), decoding is unambiguous: we simply read the string two bits at a time.<sup>2</sup>

Other finite objects that programs work with are encoded similarly. For example, a directed graph  $G$  on vertices  $\{v_1, \dots, v_n\}$  can be represented by its  $n \times n$  adjacency matrix  $A$ , where  $A_{i,j} = 1$  if there is an edge  $v_i \rightarrow v_j$  and 0 otherwise. Flattening this matrix row by row yields a binary word of length  $n^2$ . For instance, the graph on 3 vertices with edges  $v_1 \rightarrow v_2$  and  $v_2 \rightarrow v_3$  and its corresponding adjacency matrix are:



$$A = \begin{pmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 0 & 0 & 0 \end{pmatrix},$$

which is encoded by the word 010001000.

## 1.2 Computable functions

Fix your favourite general-purpose programming language (we will use Python in these notes). Consider programs that take one input word and produce one output word over some alphabet  $\Sigma$ . As explained above, multiple arguments, numbers, graphs, or structured data can always be encoded into a single word in  $\Sigma^*$  (and decoded from the output).

<sup>1</sup>If you like algebra, you may note that  $\Sigma^*$  with concatenation and  $\varepsilon$  is thus the free monoid generated by  $\Sigma$ .

<sup>2</sup>In ASCII, a standard encoding in real computers, each character is an 8-bit byte.

Thus, each program  $\pi$  naturally computes a (partial!) function  $f_\pi: \Sigma^* \rightharpoonup \Sigma^*$ .<sup>3</sup> This is how it is defined.

**Definition 2** (Function computed by a program  $\pi$ ). *Given  $u \in \Sigma^*$ , run  $\pi$  on  $u$ .*

- *If  $\pi$  halts (i.e., terminates, concludes the execution), then  $f_\pi(u) = v$ , where  $v \in \Sigma^*$  is the word it outputs.*
- *If  $\pi$  does not halt, then  $f_\pi(u)$  is undefined.*

In real life, programs may also crash with an error. In theory, we treat crashes as equivalent to non-halting.

**Definition 3** (Computable function). *A partial function  $f: \Sigma^* \rightharpoonup \Sigma^*$  is called **computable** if there exists a program  $\pi$  such that  $f = f_\pi$ .*

Note that when  $f: \Sigma^* \rightarrow \Sigma^*$  is **total**, i.e., defined on all words in  $\Sigma^*$ , the corresponding program  $\pi$  must halt on every input.

Let us consider some examples of computable functions.

**Example 4.** The following functions are all computable:

- *reverse*: given  $w \in \Sigma^*$ , outputs its reverse (e.g.,  $\text{reverse}(abc) = cba$ );
- *convert*: interprets  $w \in \{0, 1, \dots, 9\}^*$  as an integer  $n$ , outputs a word in  $\{0, 1, 2, 3\}^*$ —the same number, but in 4-ary notation;
- *double*: interprets  $w$  as an integer  $n$ , outputs  $2n$ ;
- *multiply*: given a pair of integers  $\langle n, k \rangle$ , outputs their product  $n \cdot k$ ;
- *shortestPath*: given a directed graph and two vertices  $\langle G, s, t \rangle$ , outputs the length of the shortest path from  $s$  to  $t$  (undefined if no path exists);
- *deanon*: given  $w$ , if  $w$  encodes the real name of Banksy,  $\text{deanon}(w)$  encodes the real name of Satoshi Nakamoto; for all other  $w$ , *deanon* is undefined.

### 1.3 Uncomputable functions

Are there any functions that are not computable? Yes, and it is easy to prove: there are only countably many programs, but uncountably many functions on  $\{0, 1\}^*$ . Let us see this argument in detail.

---

<sup>3</sup>Note the arrow! It is  $\rightharpoonup$ , not  $\rightarrow$ . This is how we denote that the function is *partial*.

Recall that a set  $S$  is called *countable* if its elements can be indexed by natural numbers. The set  $\{0, 1\}^*$  is, of course, countable—here is a possible indexing of words in  $\{0, 1\}^*$ :

$$\begin{aligned}
 w_0 &= \varepsilon \\
 w_1 &= 0 \\
 w_2 &= 1 \\
 w_3 &= 00 \\
 w_4 &= 01 \\
 w_5 &= 10 \\
 w_6 &= 11 \\
 w_7 &= 000 \\
 &\dots
 \end{aligned} \tag{1}$$

That is, we list the words by increasing length, and order words of the same length lexicographically. Similarly,  $\Sigma^*$  is countable for any alphabet  $\Sigma$ .

**Programs are words themselves.** Indeed, programs are just sequences of symbols that we write, and thus can be seen as words. In fact, the converse is also true: we can try to execute any word as if it was a program and see what happens—a crash would just mean non-halting. So every program is a word and every word encodes some (maybe nonsensical) program. The set of all programs, and hence of all computable functions, is therefore countable.

In contrast, the set of all partial functions  $\{0, 1\}^* \rightarrow \{0, 1\}^*$  is uncountable. In fact, this is already true for the set of all functions  $\{0, 1\}^* \rightarrow \{0, 1\}$ .

**Proposition 5** (Cantor, 1891). *The set of all functions  $\{0, 1\}^* \rightarrow \{0, 1\}$  is uncountable.*

*Proof.* For clarity of notation, we use the correspondence (1) and consider the set of functions  $\mathbb{N} \rightarrow \{0, 1\}$ . Assume for contradiction that this set is countable. Then we can index all functions in this set:  $f_0, f_1, f_2, \dots$ . Each function  $f_i$  is defined by its values on each  $k \in \mathbb{N}$ . Let us write them out:

|                            |                            |                            |                            |         |
|----------------------------|----------------------------|----------------------------|----------------------------|---------|
| <b><math>f_0(0)</math></b> | $f_0(1)$                   | $f_0(2)$                   | $f_0(3)$                   | $\dots$ |
| $f_1(0)$                   | <b><math>f_1(1)</math></b> | $f_1(2)$                   | $f_1(3)$                   | $\dots$ |
| $f_2(0)$                   | $f_2(1)$                   | <b><math>f_2(2)</math></b> | $f_2(3)$                   | $\dots$ |
| $f_3(0)$                   | $f_3(1)$                   | $f_3(2)$                   | <b><math>f_3(3)</math></b> | $\dots$ |
| $\dots$                    |                            |                            |                            |         |

Consider the diagonal of this infinite table, in bold, and the function:

$$d(n) = \begin{cases} 1, & \text{if } f_n(n) = 0, \\ 0, & \text{if } f_n(n) = 1. \end{cases}$$

Then,  $d$  is a function  $\mathbb{N} \rightarrow \{0, 1\}$ , so it must appear in the above list  $f_0, f_1, f_2, \dots$  of all such functions. However, by construction,  $d$  disagrees with  $f_0$  on 0, with  $f_1$  on 1, and, in general, with  $f_n$  on  $n$ . A contradiction.  $\square$

**Corollary 6.** *There exist uncomputable functions from  $\{0,1\}^*$  to  $\{0,1\}$ .*

In fact, there are uncountably many.

And so what? No concrete example was given so far. Could it be that all *useful* functions that we may ever want to compute are countable? Unfortunately, not. Consider the following very useful *halting function*:

$$h(\langle \pi, w \rangle) = \begin{cases} 1, & \text{if } \pi \text{ halts on input } w, \\ 0, & \text{if } \pi \text{ loops forever on input } w. \end{cases}$$

Its argument is a *pair*: a program  $\pi$  and a word  $w$ , put in the  $\langle \rangle$ -brackets and separated by a comma—this is just a way to ‘pack’ two different things into one argument of a function. We assume that  $\langle, \rangle$  and the comma are all symbols of  $\Sigma$  that are not used in programs and input words.<sup>4</sup> Note that  $h$  is a total function.

**Theorem 7** (Turing, 1936). *The halting predicate is (alas!) uncomputable.*

*Proof.* Recall that we have an indexing of words in  $\Sigma^*$ :  $w_0, w_1, w_2, \dots$ , each of which can be seen as a program:  $\pi_0, \pi_1, \pi_2, \dots$ . Each  $\pi_i$  can (potentially) be run on each  $w_j$  (if  $i = j$ , we run  $\pi_i$  on its own code!)

We can imagine a table of results for the halting predicate:

|          | $w_0$                             | $w_1$                             | $w_2$                             | $w_3$                             | $\dots$  |
|----------|-----------------------------------|-----------------------------------|-----------------------------------|-----------------------------------|----------|
| $\pi_0$  | <b><math>h(\pi_0, w_0)</math></b> | $h(\pi_0, w_1)$                   | $h(\pi_0, w_2)$                   | $h(\pi_0, w_3)$                   | $\dots$  |
| $\pi_1$  | $h(\pi_1, w_0)$                   | <b><math>h(\pi_1, w_1)</math></b> | $h(\pi_1, w_2)$                   | $h(\pi_1, w_3)$                   | $\dots$  |
| $\pi_2$  | $h(\pi_2, w_0)$                   | $h(\pi_2, w_1)$                   | <b><math>h(\pi_2, w_2)</math></b> | $h(\pi_2, w_3)$                   | $\dots$  |
| $\pi_3$  | $h(\pi_3, w_0)$                   | $h(\pi_3, w_1)$                   | $h(\pi_3, w_2)$                   | <b><math>h(\pi_3, w_3)</math></b> | $\dots$  |
| $\vdots$ | $\vdots$                          | $\vdots$                          | $\vdots$                          | $\vdots$                          | $\ddots$ |

We are going to mess with the diagonal again! Suppose for contradiction that  $h$  is computed by some program  $\eta$ . We define the following *diagonal* program  $\delta$ :

```
def  $\delta(w)$ :
    if  $\eta(w, w) == 1$ :
        while True: pass    # loop forever
    else:
        return 0             # halt immediately
```

Since  $\delta$  is a valid program, we could ask what happens when  $\delta$  is run on its own code. If  $\delta$  halts when run on the word encoding  $\delta$ , then  $\eta(\delta, \delta)$  must return 1, and so  $\delta$ , according to its own code, should actually loop forever. If it does not halt,  $\eta(\delta, \delta)$  returns 0, and then  $\delta$  should halt. A contradiction.  $\square$

<sup>4</sup>This creates a problem: we need to define  $h$  also on ‘malformed’ inputs—such as  $\langle \langle, \rangle, \rangle$ —which is formally a word from  $\Sigma^*$ . But we can agree that  $h$  will be equal to 0 on such words and forget about them. What we really care about is the behaviour of  $h$  on properly formed words: a  $\langle$ , then a program, then a comma, then an input word, then a  $\rangle$ .

## Lecture 2. Decidable and enumerable languages

### 2.1 From functions to languages

Let  $f: \Sigma^* \rightarrow \{0, 1\}$  be a total function that outputs a single bit. It can be thought of as classifying input words into two categories: those on which  $f$  returns 1 (the ‘yes’ words) and those on which  $f$  returns 0 (the ‘no’ words).

This is the same as specifying a subset  $L \subseteq \Sigma^*$ —namely, the set  $L = \{w \in \Sigma^* \mid f(w) = 1\}$ .

Conversely, every set  $L \subseteq \Sigma^*$  defines a one-bit function, called its **characteristic function**:

$$\chi_L(w) = \begin{cases} 1, & \text{if } w \in L, \\ 0, & \text{if } w \notin L. \end{cases}$$

Every set  $L \subseteq \Sigma^*$  is associated with the following problem of binary classification, called the **decision problem**<sup>5</sup> for  $L$ :

*Input:*  $w \in \Sigma^*$

*Question:* Does  $w$  belong to  $L$ ?

**Definition 8.** A set of words  $L \subseteq \Sigma^*$  is called a **language**. It is **decidable** when its characteristic function is computable, and **undecidable** otherwise.

A program that computes the characteristic function  $\chi_L$  is said to **decide** the language  $L$ : it outputs 1 on words from  $L$  (accepts them) and 0 on words not from  $L$  (rejects them). Note that such a program must halt on every input—and give an answer.

We thus have a correspondence between one-bit functions and languages:

|                                           | Function $f: \Sigma^* \rightarrow \{0, 1\}$ | Language $L \subseteq \Sigma^*$      |
|-------------------------------------------|---------------------------------------------|--------------------------------------|
| <b>Task (‘problem’)</b>                   | given $w$ , compute $f(w)$                  | given $w$ , decide whether $w \in L$ |
| <b>If some <math>\pi</math> does that</b> | computable                                  | decidable                            |
| <b>This <math>\pi</math></b>              | computes the function                       | decides the language                 |

**Example 9.** The empty language  $L = \emptyset$  is decidable. It is decided by the following program:

```
def reject_all(w):
    return 0
```

**Example 10.** Any finite language  $L = \{w_0, w_1, \dots, w_n\}$  is decidable:

<sup>5</sup>The term is historical: basically, we are asked to ‘decide’ if the input word belongs to the set.

```
def decide_L(w):
    words_L = [w0, w1, ... , wn]
    if w in words_L:
        return 1
    else:
        return 0
```

**Example 11.** The language

$$\text{PRIMES} = \{w \in \{0,1\}^* \mid w \text{ is the binary encoding of a prime number}\}$$

is decidable:

```
def is_prime(w):
    n = int(w, 2)          # interpret binary string as integer
    if n < 2:
        return 0
    for d in range(2, n):
        if n % d == 0:
            return 0
    return 1
```

**Example 12.** The language

$$\text{HALT} = \{\langle \pi, w \rangle \mid \pi \text{ halts on input } w\}$$

is undecidable, by Theorem 7.

It is a tradition in the theory of computing to name languages in ALL CAPS. It might be slightly ANNOYING, but that's the tradition, and we will RESPECT IT.

## 2.2 Semantic properties of programs

We have seen that HALT is undecidable, which is sad for those who would like to *verify* that their programs behave in the expected manner—at least, halt on a given word. Are there properties of programs that one can effectively check?

Of course there are. For example, this one:

*Input:* program code

*Question:* Does this program use a variable named  $x$ ?

It should be clear that the property above is qualitatively different from HALT. The former asks about the program's *syntax*, the latter—about the *function* that the program computes, i.e. the program's *semantics*. To highlight this distinction, we introduce some terminology.

Recall that every  $w \in \Sigma^*$  can be treated as a program: feed it to Python's interpreter, specify an input, and it will do something—maybe crash immediately, maybe output Banksy's real name, or maybe output the input backwards...—it will compute some partial function  $\Sigma^* \rightarrow \Sigma^*$ .

Any language  $P \subseteq \Sigma^*$  defines a **property of programs**: a program  $\pi$  has property  $P$  if  $\pi \in P$ , and does not have it if  $\pi \notin P$ .

**Definition 13.** A property of programs  $P \subseteq \Sigma^*$  is called **semantic** if for any two programs  $\pi_1, \pi_2 \in \Sigma^*$ , if  $f_{\pi_1} = f_{\pi_2}$ , then either both  $\pi_1, \pi_2 \in P$  or both  $\pi_1, \pi_2 \notin P$ .

In other words, a semantic property is a property of *functions*: for every function, either all programs that compute it have the property, or none does.

A property  $P$  is **non-trivial** if  $P \neq \emptyset$  and  $P \neq \Sigma^*$ . That is, there are some programs that have it, and some that do not.

**Example 14.** Consider the property

$$\text{HALT-ON-EMPTY} = \{\pi \mid \pi \text{ halts on input } \varepsilon\}.$$

It is semantic:  $\pi \in \text{HALT-ON-EMPTY}$  iff  $f_\pi(\varepsilon)$  is defined. It is non-trivial: a program that always outputs 1 regardless of the input has this property, and one that loops forever on all inputs does not have it.

The following theorem is the saddest theorem in our course.

**Theorem 15** (Rice, 1953). *Every non-trivial semantic property of programs is undecidable.*

*Proof.* The proof was not covered in the lecture, so it is here just for the curious.

Let  $P \subseteq \Sigma^*$  be a non-trivial semantic property, and consider the following program:

```
def never_stop(w):
    while True: pass    # loops forever regardless of the input
```

The function that it computes,  $f_{\text{never\_stop}}$ , is the (unique) function that is undefined on all inputs.

Suppose that  $\text{never\_stop} \notin P$ . (If  $\text{never\_stop} \in P$ , consider the complement  $\bar{P} = \Sigma^* \setminus P$  instead, which is also a non-trivial semantic property. Properties  $P$  and  $\bar{P}$  are either both decidable or both undecidable, so this is without loss of generality.)

Since  $P$  is non-empty, there is some program  $\sigma \in P$ . Since  $P$  is semantic and  $\text{never\_stop} \notin P$ , we have that  $f_\sigma \neq f_{\text{never\_stop}}$ , meaning that  $\sigma$  halts on at least one input.

We reduce the Halting Problem to  $P$ . Given a pair  $\langle \pi, w \rangle$ , construct the following program  $\rho_{\pi, w}$ :

```
def rho_pi_w(x):
    pi(w)                # run pi on w (ignore x)
    return sigma(x)      # if pi halted, behave like sigma
```

If  $\pi$  halts on  $w$ , then  $\rho_{\pi,w}$  behaves exactly like  $\sigma$ , so  $f_{\rho_{\pi,w}} = f_\sigma$  and  $\rho_{\pi,w} \in P$ . If  $\pi$  does not halt on  $w$ , then  $\rho_{\pi,w}$  never reaches the second line, so  $f_{\rho_{\pi,w}} = f_{\text{never\_stop}}$  and  $\rho_{\pi,w} \notin P$ .

Therefore,  $\langle \pi, w \rangle \in \text{HALT}$  if and only if  $\rho_{\pi,w} \in P$ . If  $P$  were decidable by some program `decide_P`, we would be able to decide HALT as follows:

```
def decide_halt( $\pi$ , w):
     $\rho_{\pi,w}$  = construct( $\pi$ , w)    # construct the code of  $\rho_{\pi,w}$  as above
    return decide_P( $\rho_{\pi,w}$ )      # check whether  $\rho_{\pi,w} \in P$ 
```

This contradicts Theorem 7. □

**Example 16.** The following languages are all undecidable by Rice’s theorem:

- $\{\pi \mid \pi \text{ halts on all inputs}\}$ ;
- $\{\pi \mid \pi \text{ halts on some input}\}$ ;
- $\{\pi \mid \pi \text{ accepts the empty word}\}$ ;
- $\{\pi \mid f_\pi = f_{\pi_0}\}$  for any fixed  $\pi_0$  — the set of programs equivalent to  $\pi_0$ .

**Remark 17.** There is one hidden detail in the proof of Rice’s theorem—the *construction* of the program  $\rho_{\pi,w}$  from  $\pi$  and  $w$ . For a proper reduction, we need this construction to be computable: given the code of  $\pi$  and the word  $w$ , we must be able to algorithmically produce the code of  $\rho_{\pi,w}$ . In Python (and any real programming language), this is obvious:  $\rho_{\pi,w}$  is just a piece of source code assembled by string concatenation. Formally, a programming language (‘encoding’) is *admissible* if it has an interpreter and ensures the ability to algorithmically hard-code arguments into a given program’s source code. Rice’s theorem holds for any admissible encoding, but can fail in artificial, pathological programming languages where programs cannot be algorithmically constructed (imagine, for example, a language where *somehow* every program has an additional flag—whether it halts on the empty input or not).

## 2.3 Enumerable languages

**Definition 18.** A language  $L \subseteq \Sigma^*$  is called **recursively enumerable** (or simply **enumerable**) if there exists a program that runs indefinitely and prints a (possibly infinite) list of words, such that:

- (1) every word printed by the program belongs to  $L$ ;
- (2) every word belonging to  $L$  is eventually printed by the program (possibly with repetitions).

**Example 19.** The set  $\text{EVEN} = \{n \in \mathbb{N} \mid n = 2k, k \in \mathbb{N}\}$  is enumerable:

```
def enum_even():
    x = 0
    while True:
        print(x)
        x = x + 2
```

**Example 20.** The language  $\{0, 1\}^*$  is enumerable:

```
def enum_bin_words():
    words = [""]      # create an array with just ε
    for w in words:
        print(w)
        words.append(w + "0")
        words.append(w + "1")
```

If we replace `print(w)` with `yield w`, `enum_bin_words()` becomes an infinite generator, allowing us to write `for w in enum_bin_words():` to iterate over all words in  $\{0, 1\}^*$  just like a Python `range`.

By analogy,  $\Sigma^*$  is enumerable and can be iterated through, for any alphabet  $\Sigma$ .

**Example 21.** The language  $\text{HALT} = \{\langle \pi, w \rangle \mid \pi \text{ halts on input } w\}$  is enumerable.

Since  $\{0, 1\}^*$  is enumerable, we can enumerate (binary encodings of) all programs and all input words. But to enumerate the pairs  $\langle \pi, w \rangle \in \text{HALT}$ , we would need to know if  $\pi$  halts on  $w$ . How can we tell, if HALT is undecidable?

Imagine we had infinitely many processors, one for each pair  $\langle \pi_i, w_j \rangle$ . Then, for every  $\langle \pi_i, w_j \rangle$  we can run a process  $p_{ij}$ , all of them in parallel, and wait. When a process terminates, we print the respective pair to the output. And those processes that never terminate will never have their pairs printed—that's exactly what we want.

The hard part is how to simulate infinitely many processes on a normal computer. We will rely on the following consideration. At the lowest level, a program does not execute all at once: its execution proceeds as a discrete sequence of elementary *machine instructions*—simple primitive steps: fetching a byte from memory, performing an arithmetic operation, or jumping to another location in code. Each instruction corresponds to a single unit of execution time.

So we can simulate the ‘parallelisation’ as follows. First, we arrange the infinite two-dimensional grid of processes  $p_{ij}$  into a single linear sequence:  $p_0, p_1, p_2, \dots$  (for instance, ordering by the sum  $i + j$ ). Then, we share the processor by gradually allocating more instructions to more processes:

- On step 0, run  $p_0$  for 1 instruction.
- On step 1, run  $p_0$  for a 2nd instruction, and  $p_1$  for 1 instruction.
- On step 2, run  $p_0$  for a 3rd instruction,  $p_1$  for a 2nd instruction, and  $p_2$  for 1 instruction.
- In general, on step  $s$ , run  $p_i$  for its  $(s - i + 1)$ -th instruction for each  $i \in \{0, 1, \dots, s\}$ .

Our program `enum_halt` will maintain a list of running processes and for each store their current progress. It will iterate through this list, pushing each process forward by one step, and appending newly launched processes. When a process terminates, the respective pair  $\langle \pi_i, w_j \rangle$  is printed out.

This technique has a vintage name, *dovetailing*. The metaphor comes from carpentry: a *dovetail joint* joins two boards by interlocking alternating fingers of wood. Here, we similarly interleave small slices of execution from different processes.

**Theorem 22** (Post, 1944). *A language  $L \subseteq \Sigma^*$  is decidable if and only if both  $L$  and its complement  $\bar{L} = \Sigma^* \setminus L$  are enumerable.*

*Proof.* ( $\Rightarrow$ ) Suppose  $L$  is decided by a program `decide_L`. Let `enum_words()` be a generator for  $\Sigma^*$ , analogous to the `enum_bin_words` from Example 20. We can enumerate  $L$  simply by filtering:

```
def enum_L():
    for w in enum_words():          # iterate over all words in  $\Sigma^*$ 
        if decide_L(w) == 1:
            print(w)
```

For the complement  $\bar{L}$ , we do the exact same thing, replacing `== 1` with `== 0`.

( $\Leftarrow$ ) Let `enum_L` and `enum_not_L` enumerate  $L$  and  $\bar{L}$ , respectively. On input  $w$ , run `enum_L` and `enum_not_L` in parallel until one of them outputs  $w$  (on two processors, or on one, via dovetailing). If `enum_L` outputs  $w$ , return 1; if `enum_not_L` outputs  $w$ , return 0. Since  $w$  must belong to either  $L$  or  $\bar{L}$  (and never both), exactly one will output  $w$  at some point, so this decides  $L$ .  $\square$

**Corollary 23.** *The complement of the Halting Problem,*

$$\overline{\text{HALT}} = \{\langle \pi, w \rangle \mid \pi \text{ does not halt on } w\}$$

*is **not** enumerable.*

## Lecture 3. Turing machines

### 3.1 The 1930s: What is computation?

Until now, we have used Python to write programs—our statement of Turing’s theorem is that no Python program computes the halting function (in the terminology of Lecture 1) or decides the language HALT (in the terminology of Lecture 2). However, Turing proved his theorem in 1936, before electronic programmable computers existed and when pythons were more dangerous to work with. This deserves an explanation.

The story begins in the 1920s with David Hilbert’s grand programme (not program!) to establish the foundations of mathematics. A central question of this programme was the *Entscheidungsproblem* (the ‘decision problem’), formulated by Hilbert and Wilhelm Ackermann in 1928:

*Does there exist an effective procedure that takes as input any mathematical statement (encoded in a special notation called ‘first-order logic’) and decides whether the statement is true or false?*

No electronic computers existed at the time, so it was supposed that such a ‘procedure’ would be executed by a person with pen and paper. For a procedure to be ‘effective’, it must be applicable mechanically, by following clear instructions step by step, without creative thinking or other usual attributes of a mathematician’s work.

**Example 24.** Consider solving a quadratic equation  $ax^2 + bx + c = 0$ , where  $a \neq 0$ , over the real numbers. Compute the discriminant  $\Delta = b^2 - 4ac$ . If  $\Delta < 0$ , there are no real solutions; otherwise, evaluate  $x = \frac{-b \pm \sqrt{\Delta}}{2a}$ . This is an effective procedure: the steps are explicit and require no mathematical insight or intuition.

Hilbert believed mathematics was decidable: that any well-posed mathematical question could be decided mechanically. To prove this belief rigorously—or to disprove it, as eventually happened—one needed a formal definition of an effective procedure, i.e. of an *algorithm*.

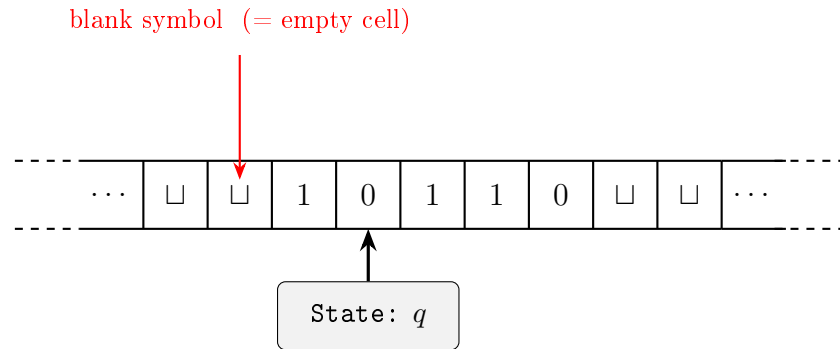
Several definitions of an algorithm were proposed in the 1930s. Although developed by different people, they all turned out to be equivalent in terms of which functions are computable. This is strong evidence that they capture the ‘nature of computation’. We will consider some of these definitions in our course.

Probably the most famous definition, and the first that we will study (although not the first historically), was given by Alan Turing in 1936.

### 3.2 The Turing machine

Informally speaking, a Turing machine is a device consisting of three components:

1. An **infinite tape** divided into discrete cells, extending indefinitely in both directions. Each cell can hold exactly one symbol from a finite **tape alphabet**  $\Gamma$ .
2. A **read/write head** that points to a single cell on the tape at any moment.
3. A **finite control** that stores the internal state  $q$  from a finite set of states  $Q$ .



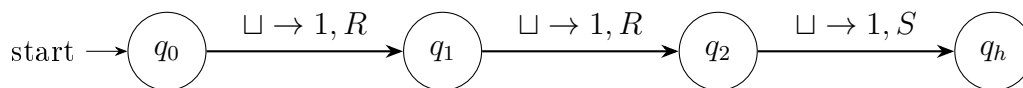
The machine works in steps. At each step, it reads the symbol written in the current cell, and then, depending on that symbol and on the state it is in:

1. overwrites the cell with a new symbol;
2. moves one cell to the right, or to the left, or stays put;
3. switches to another state (of the finitely many available).

A concrete machine is defined by specifying its behaviour for each combination of the observed symbol and current state, and a *halting state*, upon entering which the machine turns off.

**Example 25.** Consider a machine that, starting on an empty tape (where every cell contains  $\sqcup$ ), prints three 1s and stops.

We can achieve this using four states:  $q_0$  (initial),  $q_1$ ,  $q_2$ , and a halting state  $q_h$ . The machine starts in  $q_0$  scanning a blank cell, overwrites it with 1, moves right, and switches to  $q_1$ . In  $q_1$ , it writes another 1, moves right, and switches to  $q_2$ . Finally, in  $q_2$ , it writes a third 1 and enters the halting state  $q_h$ :



Here, a label  $\sqcup \rightarrow 1, R$  means “if reading  $\sqcup$ , write 1 and move right”, while  $S$  means “stay put”.

Formally, we define a Turing machine as follows.

**Definition 26.** A *Turing machine* (TM) is a tuple of six components:

$$M = \langle Q, \Sigma, \Gamma, \delta, q_0, q_h \rangle$$

where:

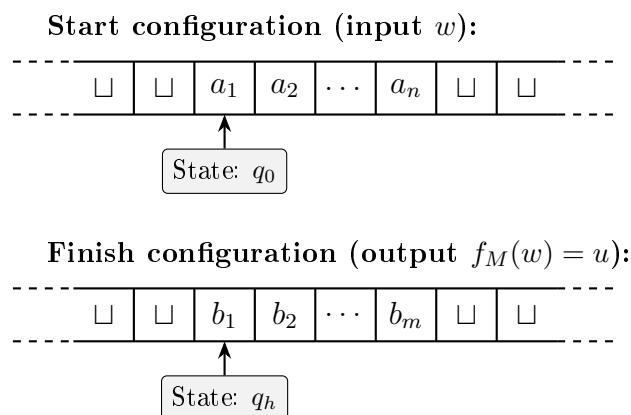
- $Q$  is a finite, non-empty set of **states**;
- $\Sigma$  is an **input alphabet**;
- $\Gamma \supseteq \Sigma$  is a **tape alphabet** with a special **blank symbol**  $\sqcup \in \Gamma \setminus \Sigma$ ;
- $\delta: Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R, S\}$  is the **transition function**;
- $q_0 \in Q$  is the **initial state**;
- $q_h \in Q$  is the **halting state**;

The webpage <https://turingmachine.io/> contains several examples and visualisations of Turing machines, and allows you to program your own. Try it!

### 3.3 Computing functions and deciding languages on Turing machines

A Turing machine  $M$  computes a partial function  $f_M: \Sigma^* \rightarrow \Sigma^*$ , defined as follows:

1. Write the input word  $w = a_1 a_2 \dots a_n$  in  $n$  consecutive cells, with blanks in all other cells.
2. Place  $M$ 's head on the first symbol of  $w$  (or on a blank cell if  $w = \varepsilon$ ).
3. Start  $M$  from its initial state  $q_0$  and let it run:
  - if  $M$  ever reaches the halting state (i.e. *halts*), and the resulting tape contains a word  $u \in \Sigma^*$  with all other cells blank, then  $f_M(w) = u$ , provided that the head is observing the first symbol of  $u$  when  $u \neq \varepsilon$ , or a blank cell when  $u = \varepsilon$ ;
  - in all other cases (no halting, symbols not from  $\Sigma \cup \{\sqcup\}$  on the tape, more than one word on the tape),  $f_M(w)$  is undefined.



Here  $u = b_1b_2 \dots b_m$ ; when  $u = \varepsilon$ , the finish configuration contains only blanks.

Clearly, if  $\{0, 1\} \subseteq \Sigma$ , we can program a machine  $M$  computing some function  $f_M: \Sigma^* \rightarrow \{0, 1\}$ . This machine will classify words of  $\Sigma^*$  into two classes and **decide** the language  $\{w \in \Sigma^* \mid f_M(w) = 1\}$ , as we have seen with programs before.

**Decide-by-state version.** Consider an equivalent definition of a Turing machine, specifically designed to decide languages. Instead of one halting state it has two: an **accepting state**  $q_a$  and a **rejecting state**  $q_r$ . Run such a machine on a tape with a word  $w$ —if it ever halts, look at the state:  $q_a$  means  $f_M(w) = 1$ ,  $q_r$  means  $f_M(w) = 0$ . If it never halts, we will assume that  $f_M(w)$  is undefined.

It is widely accepted that Turing machines capture our intuitive notion of an algorithm.

**The Church–Turing Thesis.** *Every function that can be computed by an effective procedure (in the intuitive sense: by a human following explicit rules, or by any physical computing device) can be computed by a Turing machine.*

This statement is not a mathematical theorem and cannot be proved formally. It says that Turing machines provide a good formal definition of the informal notion of an algorithm.

What gives us confidence in the thesis is the remarkable convergence of different models of computation. In the 1930s, several other logicians independently proposed their own definitions of computability: Alonzo Church formulated the  $\lambda$ -calculus (we will see it in Lecture 5), Kurt Gödel and Jacques Herbrand developed recursive functions, and Emil Post introduced production systems. Despite their very different styles, all of these models turned out to compute exactly the same class of functions. Later formalisations—such as Markov normal algorithms (we will meet them in Exercise Session 4), register machines, and modern programming languages—likewise proved equivalent.

### 3.4 Examples

As before, we visualise Turing machines as directed graphs where vertices are states and edges represent transitions. A transition  $\delta(q, a) = (q', b, D)$  is drawn as an edge from  $q$  to  $q'$  labelled

$$a \rightarrow b, D$$

meaning: “read  $a$ , write  $b$ , move in direction  $D$ ”.

**Example 27.** Consider a machine with one work state  $q_0$  over  $\Sigma = \{1\}$  and  $\Gamma = \{1, \sqcup\}$ :

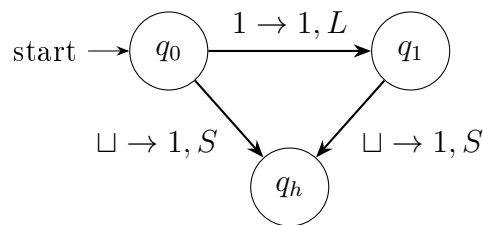


On any input word  $w$ , the machine scans rightward indefinitely without ever halting, filling the tape with 1s.<sup>6</sup>

**Example 28** (Unary +1). We represent a natural number  $n \in \mathbb{N}$  in unary as a sequence of  $n$  ones (with 0 represented by the empty word  $\varepsilon$ ).

Given  $n$  consecutive 1s on the tape, we wish to produce  $n + 1$  consecutive 1s. Rather than walking to the right end of the block, the machine can write 1 to the left of the input: if the head reads 1, it steps one cell left onto the adjacent blank, writes 1, and halts. If the input is empty (the head reads  $\sqcup$ ), it writes 1 in place and halts. In both cases, the machine halts observing the first 1 of the output.

Let  $Q = \{q_0, q_1, q_h\}$  with initial state  $q_0$  and halting state  $q_h$ :



**Example 29** (Unary addition). Consider computing the sum of two numbers  $m, n \geq 1$  given as two blocks of ones separated by a + symbol:

$$\dots \sqcup \underbrace{11\dots 1}_m + \underbrace{11\dots 1}_n \sqcup \dots$$

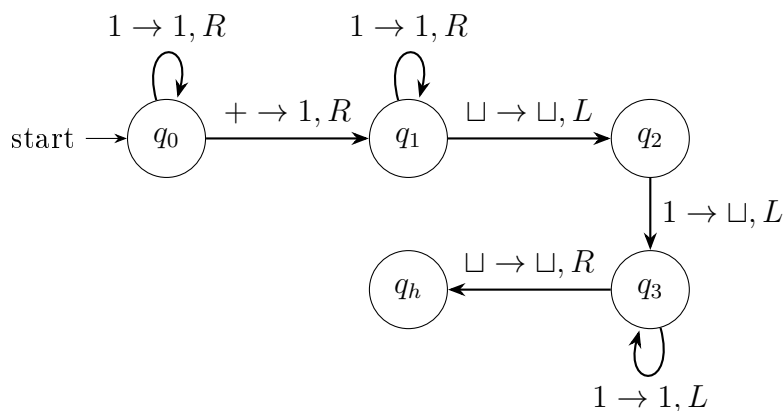
The procedure is as follows:

1. Walk right across the first block of 1s until reaching the + symbol.
2. Replace + with 1, joining the two blocks into a single block of  $m + 1 + n$  ones.
3. Walk to the right end of this block, step back onto the rightmost 1, and erase it (write  $\sqcup$ ), leaving  $m + n$  ones.
4. Walk left across all 1s until reaching the blank cell  $\sqcup$  to the left of the block, then step one cell right onto the first 1 and halt.

Here is the machine  $Q = \{q_0, q_1, q_2, q_3, q_h\}$  with initial state  $q_0$  and halting state  $q_h$ :

---

<sup>6</sup>Strictly speaking, we should also specify a halting state  $q_h$ —by definition, every machine must have it. But in this machine, it is *unreachable*—no sequence of transitions starting from the initial state ever leads to it, so we omit it from the diagram.

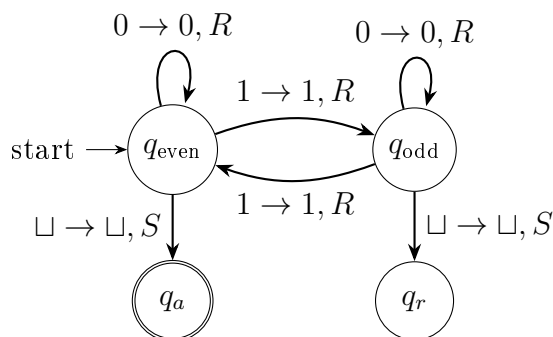


**Example 30** (Parity checker). Let us construct a machine with accepting and rejecting states that decides whether a binary string  $w \in \{0, 1\}^*$  contains an *even* number of 1s.

We use two work states to track the parity of the number of 1s seen so far:  $q_{\text{even}}$  (initial state) and  $q_{\text{odd}}$ . While scanning  $w$ , reading 0 keeps the state unchanged, while reading 1 toggles between  $q_{\text{even}}$  and  $q_{\text{odd}}$ .

When the machine hits the blank cell  $\square$  at the end of the input word:

- from  $q_{\text{even}}$ , it transitions to the accepting state  $q_a$ ;
- from  $q_{\text{odd}}$ , it transitions to the rejecting state  $q_r$ .



### 3.5 The universal Turing machine

A Turing machine is a single-purpose device: its behaviour on every input is fixed by its transition function. One machine computes binary addition; another decides whether a binary word represents an even number. A computer, however, is more versatile: we program it to perform different tasks. This familiar idea was far from obvious in the 1930s, and we owe its mathematical formulation to Alan Turing.

Turing imagined a universal machine that can be programmed to simulate any other Turing machine. Before we define it, let us remove an apparent complication: different Turing machines may have different tape alphabets.

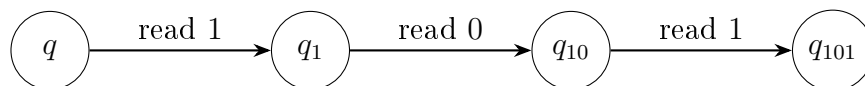
**Simulating an alphabet in binary.** Let  $M$  have tape alphabet  $\Gamma$ . Choose  $k$  such that  $2^k \geq |\Gamma|$ , and assign a different block of  $k$  bits to every symbol of  $\Gamma$ . For example, if

|                                             |        |          |     |     |     |     |     |     |     |
|---------------------------------------------|--------|----------|-----|-----|-----|-----|-----|-----|-----|
| $\Gamma = \{\sqcup, a, b, c, d, e, 0, 1\},$ | symbol | $\sqcup$ | $a$ | $b$ | $c$ | $d$ | $e$ | $0$ | $1$ |
|                                             | code   | 000      | 001 | 010 | 011 | 100 | 101 | 110 | 111 |

then three cells of  $M$  are represented by three blocks on a binary tape:

|       |       |       |
|-------|-------|-------|
| $a$   | $e$   | $c$   |
| 0 0 1 | 1 0 1 | 0 1 1 |

The simulating machine reads one block at a time. For every state  $q$  of  $M$ , it has separate states  $q_1, q_{10}, q_{101}, \dots$  that record both  $q$  and the prefix read so far. While reading the block 101, for instance, it follows:



In state  $q_{101}$ , the simulator knows both that  $M$  is in state  $q$  and that the scanned symbol is  $e$ . It can therefore apply the transition prescribed for  $(q, e)$ . Further intermediate states remember the new symbol and direction while the simulator rewrites the three bits and moves to the appropriate neighbouring block.

For  $k$  fixed in advance, there are only finitely many positions within a block and finitely many prefixes to remember, so all this information fits into finitely many states. We may therefore replace any fixed tape alphabet by a binary one (note that even the blank symbol  $\sqcup$  can be encoded as  $k$  consecutive 0s, and thus not needed—an empty tape is just a tape filled with 0s).

Consequently, we may freely use finitely many auxiliary tape symbols. For example, we can mark a symbol  $a$  by replacing it with a decorated copy  $\hat{a}$ , or add separators and flags: these are merely extra symbols, and a larger fixed block size encodes them all in binary.

**Encoding a machine.** It now suffices to encode machines whose tape alphabet is  $\{0, 1\}$ , with 0 serving as the blank symbol. Suppose such a machine  $M$  has  $m$  states. Number them

$$q_0, q_1, \dots, q_{m-1},$$

renaming if necessary so that  $q_0$  is initial and  $q_{m-1}$  is halting. Begin the code with  $m$  ones followed by a zero. From this prefix, a decoder learns  $m$  and hence the fixed number of bits needed to write any state number.

Next list the two transitions from each state, first the one for reading 0 and then the one for reading 1. A transition record contains the fixed-width binary number of the new state, the bit to write, and two bits for the direction (say, 00 for  $L$ , 01 for  $R$ , and 10 for  $S$ ). There are exactly  $2m$  such records, all of the same known length. Their concatenation after the prefix is therefore a binary word which unambiguously encodes the machine  $M$ . We will denote this word by  $[M]$ .

The input  $w$  is already binary. We encode the pair consisting of  $M$  and  $w$ , for example, as a word  $\langle [M], w \rangle$ , where  $[M]$  is the binary word above, while  $\langle, \rangle$  and the comma are special symbols that can also be encoded in binary. The particular convention is unimportant; what matters is that a fixed Turing machine can decode it and determine where  $[M]$  ends and  $w$  begins.

**Simulating an encoded machine.** A **universal Turing machine**  $U$  receives  $\langle M, w \rangle$  and maintains three pieces of information on its tape:

- the description  $[M]$ ;
- the current state of the simulated machine;
- an encoding of its tape, with one marked cell (say  $\hat{0}$  instead of 0, or  $\hat{1}$  instead of 1) indicating the position of the head of  $M$ .

Initially these contain  $[M]$ ,  $q_0$ , and the input  $w$ . To simulate one step,  $U$  reads the marked cell, scans  $[M]$  for the transition with the current state and symbol, writes the new symbol, moves the marker to the neighbouring cell, and records the new state. If the marker moves beyond the represented part of the tape,  $U$  adds a cell containing the blank symbol 0. Each operation is a finite combination of scans, comparisons, and copies, so a fixed Turing machine can perform it. Repeating this procedure simulates every step of  $M$ .

**Theorem 31** (Turing, 1936). *There exists a Turing machine  $U$  such that, for every Turing machine  $M$  and input  $w$ ,*

$$U(\langle M, w \rangle) = M(w).$$

*That is,  $U$  halts exactly when  $M$  halts, and in that case produces the same output.*

The universal machine is the theoretical model of a general-purpose computer:  $[M]$  is a program,  $w$  is its input, and  $U$  executes the program. We can now restate Theorem 7 for Turing machines:

**Theorem 32** (Turing, 1936). *The language*

$$\text{HALT}_{\text{TM}} = \{\langle M, w \rangle \mid M \text{ halts on } w\}$$

*is undecidable.*

## Lecture 4. From calculating machines to modern computers

### 4.1 Plan and destination (about 4 minutes)

- Opening question: *What makes a computer a computer?*
  - Ability to calculate quickly? An abacus can help with that.
  - Ability to operate automatically? Clocks and special-purpose calculators can do that.
  - Ability to follow a program? The Jacquard loom and many early machines could do that.
  - Ability to become, through its program, any computing machine we know how to describe? This is the decisive idea of *universality*.
- Central storyline for the lecture:
  - represent a problem physically;
  - mechanise a sequence of operations;
  - encode the sequence as a program;
  - treat the program itself as data;
  - build one machine that can imitate indefinitely many special-purpose machines;
  - hide the resulting memory bottleneck behind caches, parallelism, and specialised processors.
- Learning goals:
  - place the principal machines and people in historical order;
  - distinguish *automatic*, *programmable*, *electronic*, *general-purpose*, and *stored-program*;
  - explain the universal-machine idea without relying on the formalism of Lecture 3;
  - compare Harvard and von Neumann architectures;
  - tell an intuitive story of what happens inside a present-day computer when a program runs.
- Recurring blackboard line: **Where are the instructions? Where is the data? Can the machine change either?**

### 4.2 Before “the computer”: externalising arithmetic (about 6 minutes)

- Abacus / counting board:
  - stores a number as a physical arrangement of counters;
  - the human supplies the algorithm and moves the counters;
  - useful distinction: it is a *memory and manipulation aid*, not an autonomous program-running machine;

- origins are uncertain; forms of counting board and abacus appeared in several cultures. Avoid claiming a single inventor.<sup>7</sup>
- Optional one-minute curiosity—the Antikythera mechanism (first century BCE):
  - a hand-driven geared model that predicted astronomical positions and eclipses;
  - a striking example of a sophisticated *analogue, special-purpose* computer;
  - its mechanism embodies one particular model of the heavens: changing the problem means rebuilding the machine.<sup>8</sup>
- Seventeenth-century mechanical calculators (very briefly):
  - Pascal’s calculator (1640s): gear arithmetic, chiefly addition and subtraction;
  - Leibniz’s stepped reckoner: sought to mechanise all four arithmetic operations;
  - conceptual step: the operator supplies inputs, but the mechanism performs increasingly long pieces of arithmetic.
- Theme to retain: a machine may be ingenious and automatic while remaining tied to one narrow family of tasks.

### 4.3 Instructions become a physical object (about 5 minutes)

- Jacquard’s early nineteenth-century loom:
  - a chain of punched cards controls which threads are raised;
  - change the cards, change the woven pattern, without rebuilding the loom;
  - the cards are an *external program*: instructions are physically separate from the material being worked on.
- Use a punched card as the first tangible image of software:
  - hole / no hole is a discrete symbol;
  - a sequence of cards is a sequence of commands;
  - repetition can be made literal by joining tape into a loop.
- Long after Jacquard, Herman Hollerith used punched cards for *data*, not loom instructions, in tabulating the 1890 US census:
  - the same medium can encode commands in one system and facts about people in another;
  - Hollerith’s company is one ancestor of IBM.<sup>9</sup>

---

<sup>7</sup>Smithsonian National Museum of American History, *The Abacus and the Numeral Frame*: <https://americanhistory.si.edu/collections/object-groups/the-abacus-the-numeral-frame-and-counters>.

<sup>8</sup>Science Museum, *Ancient Greek Computing*: <https://blog.sciencemuseum.org.uk/ancient-greek-computing/>.

<sup>9</sup>Smithsonian, *Punch Cards for Data Processing*: <https://americanhistory.si.edu/collections/object-groups/punch-cards/punch-cards-data-processing>; IBM, *The punched card tabulator*: <https://www.ibm.com/history/punched-card-tabulator>.

## 4.4 Babbage and Lovelace: the computer imagined in brass (about 11 minutes)

- Charles Babbage (1791–1871) and the Difference Engine, proposed in 1822:
  - motivation: printed mathematical tables used in navigation, astronomy, finance, and engineering were laborious and error-prone;
  - use the method of finite differences to calculate and print tables automatically;
  - special-purpose in its intended operation, despite formidable mechanical sophistication;
  - vivid detail: after finding yet more errors while checking tables, Babbage reportedly wished that the calculations “had been executed by steam.”<sup>10</sup>
- Babbage’s Analytical Engine, conceived from 1834:
  - a never-completed design for a decimal, general-purpose, programmable machine;
  - the *store* holds numbers; the *mill* performs arithmetic—an ancestor of memory and processor;
  - punched cards derived from the Jacquard loom specify operations and variables;
  - designs include sequencing, loops, and conditional branching;
  - crucial limitation relative to later stored-program machines: program cards remain external to the store;
  - historical caution: there is no simple continuous engineering lineage from Babbage to the electronic computer; many ideas were later reinvented.<sup>11</sup>
- Augusta Ada King, Countess of Lovelace (1815–1852):
  - daughter of the poet Lord Byron; received an unusually mathematical education for a woman of her social position;
  - met Babbage at seventeen and saw a working fragment of the Difference Engine;
  - her 1843 translation of Luigi Menabrea’s account of the Analytical Engine grew into extensive notes, including a published procedure for Bernoulli numbers;
  - call it an early published program, while noting that historians debate how to divide credit between Lovelace and Babbage;
  - deeper insight: if numbers can encode musical notes or other symbols, the engine can manipulate more than quantities—a route from *calculation* to *general symbolic computation*.<sup>12</sup>
- Blackboard comparison so far:
  - abacus: data in the device, instructions in a person;
  - Jacquard loom: instructions on cards, cloth as the object transformed;

<sup>10</sup>Computer History Museum (CHM), *A Brief History—Babbage Engine*: <https://www.computerhistory.org/babbage/history>.

<sup>11</sup>CHM, *The Engines*: <https://www.computerhistory.org/babbage/engines>.

<sup>12</sup>CHM, *Ada Lovelace*: <https://www.computerhistory.org/babbage/adalovelace>.

- Analytical Engine: external instruction cards + internal numerical store + mill;
- still missing: instructions and data in the same fast, writable memory.

#### 4.5 Turing: from one machine per task to one machine for every algorithm (about 9 minutes)

- Recall from Lecture 3: a Turing machine has a tape, head, finite control, and transition table.
- Turing’s 1936 conceptual leap:
  - the complete description of a machine is itself a finite string;
  - place an encoded machine description beside its input on a tape;
  - a single *universal machine*  $U$  reads both and simulates the described machine  $M$  on input  $w$ :

$$U(\langle M \rangle, w) = M(w).$$

- intuition: hardware no longer determines the task; an encoded description turns the same universal hardware into a calculator, compiler, game, or another computer;
- the program/data distinction is now a matter of *interpretation*: both are symbols in memory.<sup>13</sup>
- What this did *not* yet provide:
  - a practical electronic circuit diagram;
  - a fast memory technology;
  - a claim that a literal tape-and-head machine is the best physical design.
- What it did provide:
  - a precise reason why one finite machine can have unboundedly many behaviours;
  - the conceptual foundation for software and interpreters;
  - the possibility of compilers, emulators, virtual machines, and even programs that inspect or generate programs.

#### 4.6 War accelerates machinery: Enigma, Bombe, and Colossus (about 9 minutes)

- Correct the tempting movie version: **Turing did not single-handedly build a general-purpose computer that “decoded Enigma.”**

---

<sup>13</sup>A. M. Turing, *On Computable Numbers, with an Application to the Entscheidungsproblem* (1936), especially the universal machine: [https://www.cs.virginia.edu/~robins/Turing\\_Paper\\_1936.pdf](https://www.cs.virginia.edu/~robins/Turing_Paper_1936.pdf); Stanford Encyclopedia of Philosophy, *Computability and Complexity*: <https://plato.stanford.edu/entries/computability/>.

- Enigma and the Bombe:
  - Polish cryptanalysts had made essential pre-war breakthroughs and shared them with Britain and France;
  - at Bletchley Park, Turing and Gordon Welchman designed the British Bombe to test possible Enigma settings quickly;
  - cryptanalysts supplied constraints and likely plaintext (a *crib*); the Bombe eliminated incompatible settings;
  - it was an electromechanical, special-purpose search machine—not a stored-program general-purpose computer;
  - Turing led Hut 8’s attack on German naval Enigma and developed Banburismus to reduce the search.<sup>14</sup>
- Bright Turing details, used sparingly:
  - the abstract logician was also a near-Olympic-level distance runner; he took fifth place in the 1947 Amateur Athletic Association marathon in 2:46:03;<sup>15</sup>
  - his wartime work was collaborative and operational as well as theoretical;
  - after prosecution for homosexuality in 1952, he accepted hormonal treatment instead of prison; he received a posthumous royal pardon in 2013.
- Do not confuse the Bombe with Colossus:
  - Colossus attacked the more complex Lorenz (“Tunny”) teleprinter cipher, not Enigma;
  - Max Newman’s codebreaking section, mathematician Bill Tutte’s analysis, and Post Office engineer Tommy Flowers’s electronic design were central;
  - operational in 1944: electronic, digital, and programmable through switches and plugboards, but special-purpose and not stored-program;
  - secrecy hid its contribution from public computer histories for decades.<sup>16</sup>
- Link back to Turing without exaggeration:
  - Bletchley showed that electronics could make large logical searches practical;
  - it trained a community in high-speed digital logic;
  - Turing contributed methods relevant to Lorenz work, but did not design Colossus.

---

<sup>14</sup>Imperial War Museums, *How Alan Turing Cracked the Enigma Code*: <https://www.iwm.org.uk/history/how-alan-turing-cracked-the-enigma-code>.

<sup>15</sup>Andrew Hodges, *Alan Turing: an introductory biography*: <https://www.turing.org.uk/publications/lausannebio.html>; King’s College, Cambridge, *Alan Mathison Turing*: <https://www.kings.cam.ac.uk/alan-mathison-turing-1912-54>.

<sup>16</sup>The National Museum of Computing, *Colossus*: <https://www.tnmoc.org/colossus>.

## 4.7 Several “first computers,” depending on the adjective (about 9 minutes)

- Warn against a pointless single-winner contest; put an adjective before *first*.
- Konrad Zuse’s Z3 (1941):
  - automatic, programmable, binary, electromechanical;
  - program read from punched film; not an electronic stored-program computer.
- Harvard–IBM Automatic Sequence Controlled Calculator / Mark I (1944):
  - Howard Aiken’s conception, engineered and built with an IBM team;
  - about 50 feet long, five tons, and electromechanical;
  - instructions read sequentially from punched paper tape; numerical values held in separate storage counters and supplied on cards or switches;
  - Grace Hopper was one of its earliest programmers and wrote its programming manual;
  - vivid details: a program loop could be a literal loop of tape, and a correction a literal patch over punched holes;
  - the famous relay-trapped moth was logged in the successor Mark II in 1947; *bug* already meant a technical fault, so it is not the word’s origin.<sup>17</sup>
- ENIAC (operational 1945, publicly unveiled 1946):
  - electronic and general-purpose, built by the University of Pennsylvania team led by J. Presper Eckert and John Mauchly;
  - dramatically faster than relay machines;
  - initially programmed by setting switches and rewiring plugboards—changing problems could take days;
  - its women programmers translated mathematical procedures into physical configurations;
  - later modified to execute instructions in a stored-program-like mode;
  - lesson: fast arithmetic is not enough if reprogramming is slow.
- Colossus, Z3, Mark I, and ENIAC each own a plausible “first” under different criteria; none initially combined every property of a modern stored-program computer.<sup>18</sup>

---

<sup>17</sup>Harvard Collection of Historical Scientific Instruments, *Mark I—Function and Language*: <https://chsi.harvard.edu/harvard-ibm-mark-1-function>, <https://chsi.harvard.edu/harvard-ibm-mark-1-language>.

<sup>18</sup>CHM, *The Neverending Quest for “First”*: <https://computerhistory.org/blog/the-neverending-quest-for-firsts/>.

## 4.8 The stored-program turning point: von Neumann, the EDVAC group, and Turing (about 11 minutes)

- The practical problem exposed by ENIAC:
  - electronic components can calculate much faster than people can rewire a machine;
  - solution: encode instructions numerically and keep them in electronic memory beside the data.
- *First Draft of a Report on the EDVAC*, circulated in June 1945 under John von Neumann’s name:
  - identifies memory, central arithmetic, central control, input, and output “organs”;
  - makes a stored sequence of instructions control the machine;
  - branching makes the next instruction depend on an intermediate result;
  - data and instructions share the same writable memory and representation;
  - the report circulated widely and became extraordinarily influential.<sup>19</sup>
- Attribution caution—why “von Neumann architecture” is useful but historically incomplete:
  - the report drew on discussions and engineering work by Eckert, Mauchly, and the wider Moore School team;
  - Herman Goldstine distributed the draft with von Neumann as sole named author, helping attach one name to a collective design;
  - the key stored-program idea has several roots; historians dispute any single-inventor account.
- How Turing influenced the conceptual architecture:
  - von Neumann knew Turing’s 1936 universal-machine paper;
  - Turing had shown why machine descriptions can be represented in the same symbolic medium as data;
  - the EDVAC design turns that logical possibility into a practical organisation of electronic hardware;
  - phrase carefully: Turing supplied a foundational idea, not the EDVAC engineering design; the extent of direct influence is historically debated.<sup>20</sup>
- Turing’s own practical design:
  - joined the National Physical Laboratory in 1945 and designed the Automatic Computing Engine (ACE);
  - produced a detailed stored-program electronic computer report in 1946;

<sup>19</sup>John von Neumann, *First Draft of a Report on the EDVAC* (1945): <https://web.mit.edu/STS.035/www/PDFs/edvac.pdf>.

<sup>20</sup>Stanford Encyclopedia of Philosophy, *Turing Machines*, note on program-as-data and von Neumann’s architecture: <https://plato.stanford.edu/entries/turing-machine/notes.html>; Institute for Advanced Study, *Electronic Computer Project*: <https://www.ias.edu/electronic-computer-project>.

- his design emphasised speed, compact hardware, and sophisticated programs, but institutional delays frustrated him.<sup>21</sup>
- Bright von Neumann details:
  - Budapest child prodigy who reportedly entertained visitors by memorising telephone-directory entries;
  - worked across set theory, quantum mechanics, game theory, ballistics, meteorology, nuclear weapons, and computing;
  - also loved limericks, practical jokes, and large Princeton parties—a useful counterweight to the image of an austere logician.<sup>22</sup>
- Machines make the proposal real:
  - Manchester “Baby” (1948): first machine to execute a program held in writable electronic memory;
  - EDSAC at Cambridge (1949): an early practical stored-program computer serving users;
  - von Neumann’s IAS project disseminated reports openly; its design was copied and adapted around the world.<sup>23</sup>

## 4.9 Harvard versus von Neumann architecture (about 8 minutes)

- First clarify chronology and terminology:
  - the Harvard Mark I (1944) preceded the EDVAC report (1945);
  - *Harvard architecture* is a later architectural label inspired by the separation visible in the Mark I, not the title of Aiken’s rival theoretical manifesto.
- Harvard organisation:
  - instruction memory and data memory are physically separate;
  - separate paths allow an instruction and a datum to be fetched at the same time;
  - memories may have different widths and technologies;
  - drawback: treating code as ordinary data is less direct; capacity in one memory cannot automatically be borrowed by the other;
  - historically visible in Mark I: sequence tape for instructions, storage counters/cards/switches for numbers.
- Von Neumann organisation:

---

<sup>21</sup>The Turing Digital Archive, *A report on the detailed design of an electronic computer* (1946): <https://turingarchive.kings.cam.ac.uk/unpublished-manuscripts-and-drafts-amtc/amt-c-32>; National Physical Laboratory, *Alan Turing*: <https://www.npl.co.uk/about-us/history/famous/alan-turing>.

<sup>22</sup>Institute for Advanced Study, *John von Neumann: Life, Work, and Legacy*: <https://www.ias.edu/von-neumann>.

<sup>23</sup>CHM, *The Stored Program*: <https://www.computerhistory.org/revolution/story/87>.

- one memory/address space holds instructions and data;
  - one communication path is shared in the simple textbook version;
  - advantages: simple, flexible stored programs; code can be loaded, copied, generated, or modified like data;
  - drawback: instructions and data compete for the path between processor and memory—the *von Neumann bottleneck*.
- One-sentence comparison:
    - Harvard asks, “How can we keep instruction traffic and data traffic apart?”
    - von Neumann asks, “What becomes possible if instructions are data too?”
  - Avoid declaring a modern laptop purely one or the other:
    - most general-purpose systems present a unified address space to software;
    - many processors have separate level-1 instruction and data caches and paths, then unified lower caches and RAM;
    - this hybrid is often called *modified Harvard architecture*.<sup>24</sup>

## 4.10 What happens inside a computer now? (about 10 minutes)

- Running example: the user opens a photo and increases its brightness.
- Before the CPU executes the interesting instruction:
  - program bytes live persistently on SSD/flash;
  - the operating system maps or loads required code and data pages into RAM;
  - to the program, both instructions and pixels are numbered bytes in a virtual address space—the stored-program inheritance;
  - security permissions normally mark code executable and image data non-executable: same kind of storage, different allowed uses.
- A deliberately simplified fetch–decode–execute cycle:
  - a *program counter* gives the address of the next instruction;
  - instruction bytes are fetched, usually from the instruction cache rather than all the way from RAM;
  - the control logic decodes the bits: for example, load pixels, add or multiply, store results, or branch;
  - tiny, fast *registers* hold immediate operands;
  - arithmetic/logic or vector units perform the operation;

---

<sup>24</sup>Arm, *Cortex-R Series Programmer’s Guide*, Section 8.3: <https://documentation-service.arm.com/static/684be32a3f793d5d7b223563>.

- the result is placed in a register/cache and eventually written back to RAM;
  - the program counter advances or changes at a branch, and the cycle continues billions of times per second.
- Why this is still recognisably von Neumann:
    - stored instructions direct an arithmetic/control unit;
    - instructions and data ultimately inhabit a common memory system;
    - input and output move information between the machine and the world.
  - Why the physical machine is much less simple than the textbook diagram:
    - registers and several cache levels hide slow main memory;
    - separate instruction/data caches borrow the Harvard idea;
    - pipelines overlap stages; superscalar and out-of-order execution work on several instructions at once;
    - branch predictors guess which instructions will be needed;
    - several CPU cores run instruction streams in parallel;
    - GPUs apply the same operations to many pixels or numbers; neural accelerators and other devices specialise further;
    - direct memory access and interrupts let devices exchange data without the CPU micro-managing every byte.
  - The universal-machine view of the whole stack:
    - Python source is data to the editor;
    - it is a program to the Python interpreter;
    - the interpreter is data loaded by the operating system and machine code to the CPU;
    - a virtual machine or emulator can make one physical machine imitate another;
    - universality concerns *what can be computed in principle*; architecture concerns *how to compute it reliably, cheaply, and quickly*.

#### 4.11 Close: the three ideas to carry forward (about 3 minutes)

- **Representation:** stones, gears, holes, electrical voltages, and magnetic states can all stand for numbers or symbols.
- **Universality:** Turing showed that one machine can interpret the encoded description of any other effective machine.
- **Stored program:** the EDVAC/IAS tradition made program-as-data the practical organising principle of electronic computers.
- Final synthesis:

- conceptually, a modern computer is Turing’s universal interpreter made physical;
  - at the system level, it remains largely von Neumann: programs and data share an addressable memory world;
  - near the processor, it often becomes Harvard-like: separate instruction and data paths keep the machine fed;
  - around the CPU, specialised accelerators trade some universality for speed and energy efficiency;
  - the history is not a relay race of lone geniuses, but a convergence of mathematical ideas, mechanical craft, electronics, war, institutions, and large collaborative teams.
- Exit question: *If code and data are both bits, what makes some bits an instruction?*
    - expected answer: not their physical substance, but where they are fetched from, how the processor interprets them, and what the system permits.

## 4.12 Optional cuts and expansions

- If short on time:
  - omit Pascal and Leibniz;
  - reduce the “several firsts” section to Mark I, Colossus, and ENIAC;
  - keep Babbage/Lovelace, Turing, stored program, the architecture comparison, and the modern walkthrough.
- If there is extra time:
  - demonstrate one line of punched tape or a literal paper loop;
  - ask students to classify machines along five axes: automatic / programmable / electronic / general-purpose / stored-program;
  - trace a tiny three-instruction program through program counter, registers, ALU, and memory;
  - discuss self-modifying code, just-in-time compilation, or the security meaning of non-executable data pages;
  - connect the von Neumann bottleneck to GPUs and machine-learning accelerators.